

1. Classes and Objects

(A) Class Fundamentals :

- Class एक user defined data type होता है जो किसी set of objects के common properties (attributes) और behaviors (methods) को define करता है।
- Class एक blueprint या template की तरह होता है, जिसके द्वारा objects create किए जाते हैं।
- Object, class का instance होता है।
- Class में variables को fields (या attributes) और functions को methods कहा जाता है।
- Class के माध्यम से code reusability, data hiding और modularity प्राप्त होती है।

Class Structure :

```
class ClassName {  
    // fields (attributes)  
    dataType field1;  
    dataType field2;  
    ...  
    // methods (behaviors)  
    returnType methodName() {  
        // statements  
    }  
    ...  
}
```

Example :

```
class Student {  
    // attributes  
    int rollNo;  
    String name;  
    float marks;  
  
    // method  
    void display() {  
        System.out.println("Roll No: " + rollNo);  
        System.out.println("Name: " + name);  
        System.out.println("Marks: " + marks);  
    }  
}
```

Object Creation :

Student (Class) $\xrightarrow{\text{new}}$ s1 (Object)

```
Student s1 = new Student();  
s1.rollNo = 101;  
s1.name = "Amit";  
s1.marks = 85.5f;  
s1.display();
```

(B) Constructors :

- Constructor एक special member function होता है जिसका नाम class के नाम के समान होता है।
- Constructor का उपयोग object को initialize करने के लिए किया जाता है।
- Constructor का return type नहीं होता, यहाँ तक कि void भी नहीं।
- Constructor को object create करते समय automatic call किया जाता है।

Constructor के Rules :

- (1) इसका नाम class name के समान होता चाहिए।
- (2) इसका return type नहीं होता (not even void).
- (3) Constructor को call नहीं किया जाता, यह automatic call होता है।
- (4) एक class में multiple constructors हो सकते हैं (overloading allowed).

Types of Constructors :

1. Default Constructor
(No-argument constructor)

2. Parameterized Constructor
(With-argument constructor)

1. Default Constructors :

- जब constructor में कोई parameter नहीं होता।
- यदि हम कोई constructor नहीं बनाते, तो compiler automatic default constructor provide करता है।

Example :

```
class Demo {  
    int x;  
    String name;  
  
    // default constructor  
    Demo() {  
        x = 0;  
        name = "Unknown";  
    }  
    void show() {  
        System.out.println(x + " " + name);  
    }  
}
```

2. Parameterized Constructor :

- जब constructor में एक या अधिक parameters होते हैं।
- यह object create करते समय values receive करता है।

Example :

```
class Student {  
    int rollNo;  
    String name;  
    float marks;  
  
    // parameterized constructor  
    Student(int r, String n, float m) {  
        rollNo = r;  
        name = n;  
        marks = m;  
    }  
    void display() {  
        System.out.println(rollNo + " " + name + " " + marks);  
    }  
}
```

Constructor Overloading :

- Same class में different parameter list के साथ multiple constructors बनाए जा सकते हैं।

Example :

```
class Point {  
    int x, y;  
    Point() {  
        x = 0; y = 0;  
    }  
    Point(int a, int b) {  
        x = a; y = b;  
    }  
    Point(int a) {  
        x = a; y = a;  
    }  
}
```

Note :

- ✓ Class blueprint होता है, Object उसका real world instance होता है।
- ✓ Constructor object को initialize करने के लिए उपयोगी होता है।
- ✓ Proper use of constructors program को reliable और readable बनाता है।

Declaring Objects (Object & Object Reference):

1. Object क्या होता है ?

- Object किसी class का real world instance होता है।
- जब class के blueprint (template) को memory में allocate किया जाता है, तब object बनता है।
- Object में class के attributes के लिए memory space और methods को call करने की capability होती है।

महत्वपूर्ण Point :

- Class = Blueprint / Design
- Object = Real Entity / Instance
- Reference = उस object का address जिसे हम variable में store करते हैं।

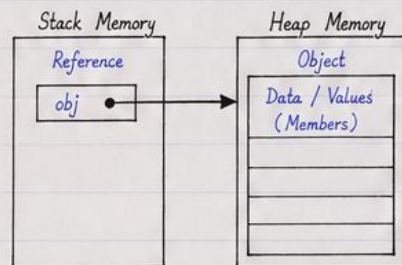
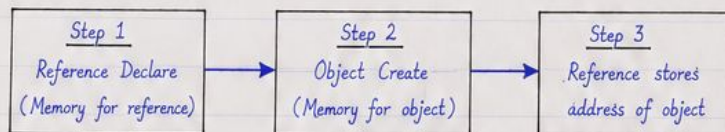
2. Object Reference क्या होता है ?

- Java में object को directly name से access नहीं किया जाता, बल्कि एक reference variable के द्वारा access किया जाता है।
- यह reference variable उस object का address store करता है।
- इस reference को ही Object Reference कहते हैं।

Syntax :

```
ClassName referenceName; // Declaration  
referenceName = new ClassName(); // Object Creation
```

3. Object Declare और Create करने की प्रक्रिया :



4. Example :

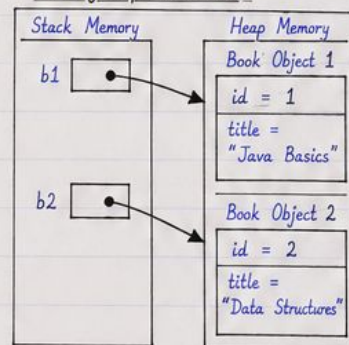
(i) Simple Example :

```
class Student {  
    int rollNo;  
    String name;  
}  
public class Test {  
    public static void main(String[] args) {  
        Student s; // Step 1: Reference Declare  
        s = new Student(); // Step 2: Object Create  
        // Step 3: Reference stores address  
        s.rollNo = 101;  
        s.name = "Amit";  
        System.out.println(s.rollNo + " " + s.name);  
    }  
}
```

(ii) Multiple Objects :

```
class Book {  
    int id;  
    String title;  
}  
public class TestBook {  
    public static void main(String[] args) {  
        Book b1 = new Book(); // Declare + Create  
        Book b2 = new Book(); // Declare + Create  
        b1.id = 1;  
        b1.title = "Java Basics";  
        b2.id = 2;  
        b2.title = "Data Structures";  
    }  
}
```

Memory Representation :



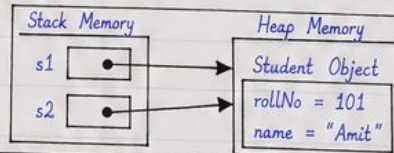
5. Key Points :

- ✓ Reference variable और object दो अलग चीजें हैं।
- ✓ Reference variable stack memory में store होता है।
- ✓ Object heap memory में store होता है।
- ✓ new keyword के द्वारा object heap memory में create होता है।
- ✓ Reference variable उस object का address रखता है।
- ✓ एक reference variable एक ही समय में एक ही object को point कर सकता है, लेकिन एक object को कई reference variables point कर सकते हैं।

Example (Multiple Reference to Same Object):

```
Student s1 = new Student();  
Student s2 = s1; // s2 भी उसी object को  
point करेगा जिसे s1 कर रहा है।
```

Diagram :



Note : Object oriented programming (OOP) में object और reference की concept बहुत important है, क्योंकि इसी के द्वारा data hiding, encapsulation और reusability achieve होती है।

1. Creating and Accessing Variables and Methods :

- Object create करने के बाद उस object के variables (fields) और methods (functions) को access किया जाता है।
- Variables को '.' (dot) operator के द्वारा access करते हैं।
- Methods को call करके execute किया जाता है।

Example :

```
class Student {
    int rollNo;
    String name;

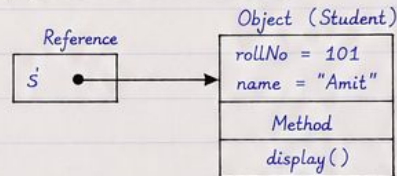
    void display() {
        System.out.println("Roll No : " + rollNo);
        System.out.println("Name : " + name);
    }
}

// Main class
public class Test {
    public static void main(String[] args) {
        Student s = new Student(); // Object creation
        s.rollNo = 101; // Accessing variables
        s.name = "Amit"; // Accessing variables
        s.display(); // Accessing method
    }
}
```

Explanation :

- `s = new Student();` → Object create होता है।
- `s.rollNo` और `s.name` द्वारा variables में value store होती है।
- `s.display();` द्वारा method call होता है और output प्राप्त होता है।

Object and Members Access :



Reference variable 's' object के members (variables और methods) को access करता है।

2. Static and Non-Static Variables / Methods :

- Static members class के नाम से belong करते हैं, object के बिना access हो सकते हैं।
- Non-static members object के साथ belong करते हैं, इन्हें access करने के लिए object जरूरी होता है।

Feature	Static (Class) Members	Non-Static (Instance) Members
Belongs to	Class	Object / Instance
Memory	Method Area (एक copy सभी objects के लिए)	Heap Memory (हर object के लिए अलग copy)
Access	ClassName.memberName	objectRef.memberName
Created	जब class load होती है	जब object create होता है
Can access	केवल static members को directly access कर सकते हैं	static और non-static दोनों को access कर सकते हैं
Example	static int count; , static void show();	int id; , void display();

Example (Static) :

```
class Counter {
    static int count = 0;
    static void showCount() {
        System.out.println("Count = " + count);
    }
}

public class Test {
    public static void main(String[] args) {
        Counter.count = 10; // Access static variable
        Counter.showCount(); // Call static method
    }
}
```

Example (Non-Static) :

```
class Student {
    int rollNo;
    void display() {
        System.out.println("Roll No : " + rollNo);
    }
}

public class Test {
    public static void main(String[] args) {
        Student s1 = new Student(); // Object create
        s1.rollNo = 101; // Access variable
        s1.display(); // Call method
    }
}
```

Note :

- ✓ Static members memory और time बचाते हैं।
- ✓ Non-static members हर object के लिए unique data provide करते हैं।

3. Defining Packages :

- Package एक group of related classes और interfaces का collection होता है।
- Package का use code organization, reusability और naming conflict को avoid करने के लिए किया जाता है।
- Package के classes को use करने के लिए import statement का उपयोग करते हैं।

Advantages of Packages :

- Code को well organized बनते हैं।
- Naming conflict को avoid करते हैं।
- Code reusability बढ़ती है।
- Access control provide करते हैं (public, default, protected, private).

Creating a Package :

```
// File Name : MyPackage/Student.java
package MyPackage;

public class Student {
    public void display() {
        System.out.println("Hello from MyPackage");
    }
}
```

Using a Package :

```
// File Name : Test.java
import MyPackage.Student; // Import statement

public class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

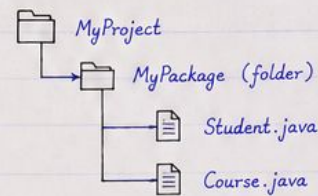
Important Points :

- ✓ Package name हमें छोटे अक्षरों में होना चाहिए।
- ✓ एक file में एक ही package statement होना चाहिए और वह सबसे पहली line में होना चाहिए (comments को छोड़कर)।
- ✓ import statement package के classes को use करने के लिए किया जाता है।

1. Creating and Accessing a Package :

- Package का use code को organize करने, reusability बढ़ाने और naming conflicts को avoid करने के लिए किया जाता है।
- Package बनाई जाने पर related classes और interfaces को एक ही group में रखा जाता है।
- Package के classes और interfaces को use करने के लिए import statement का use किया जाता है।

Package Structure :



(A) Creating a Package :

1. सबसे पहले package के लिए folder (directory) बनाते हैं।
2. उस folder के अंदर .java file बनाकर उसके सबसे ऊपर package statement लिखते हैं।
3. फिर class या interface को define करते हैं।

Example :

```
// File Name : MyPackage/Student.java
package MyPackage;           // Package statement
public class Student {
    int rollNo;
    String name;
    public void display() {
        System.out.println("Roll No : " + rollNo);
        System.out.println("Name : " + name);
    }
}
```

(B) Accessing a Package :

- Package के classes और interfaces को use करने के लिए import statement लिखते हैं।
- Import के बाद class का object बनाकर उसके members (variables, methods) को access करते हैं।

Example :

```
// File Name : Test.java (Different package)
import MyPackage.Student;      // Import statement
public class Test {
    public static void main(String[] args) {
        Student s = new Student(); // Object create
        s.rollNo = 101;
        s.name = "Amit";
        s.display();             // Method call
    }
}
```

Note : Package के members को access करने के लिए पहले import करना आवश्यक होता है।

2. Importing Packages :

- Import statement किसी दूसरे package के classes और interfaces को current program में use करने के लिए किया जाता है।
- Java में तीन प्रकार के import होते हैं :

(i) Single Type Import :

किसी एक specific class या interface को import करना।

Syntax :

```
import packageName.className;
```

Example :

```
import MyPackage.Student;
import java.util.Scanner;
```

Use : केवल उन्ही classes के लिए जो program में उपयोग हो रही हैं।

(ii) On-Demand (Wildcard) Import :

किसी पूरे package की सभी classes और interfaces को एक साथ import करना।

Syntax :

```
import packageName.*;
```

Example :

```
import java.util.*;
```

Use : जब package की कई classes को use करना हो।

(iii) Static Import :

किसी class के static members (variables और methods) को direct use करने के लिए।

Syntax :

```
import static packageName.className.member;
```

Example :

```
import static java.lang.Math.PI;
import static java.lang.Math.*;
```

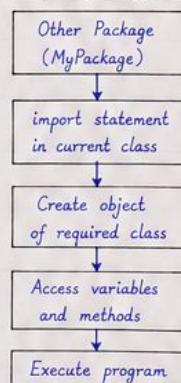
Use : Program को छोटा और readable बनाता है।

Example Program (Using Import) :

```
import java.util.Scanner;           // Single type import
import java.util.*;                // On-demand import
import static java.lang.Math.*;     // Static import

public class ImportDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter radius : ");
        double r = sc.nextDouble();
        double area = PI * r * r;    // PI is static member
        System.out.println("Area = " + area);
    }
}
```

Flow of Importing and Accessing :



Important Points :

- Package name हमेशा छोटे अक्षरों में लिखें।
- import statement हमेशा class के package declaration के बाहर (top) पर लिखें।
- Same package के classes को import करने की आवश्यकता नहीं होती।
- java.lang package की classes (जैसे String, System) auto import होती हैं।

1. Understanding CLASSPATH :

- CLASSPATH एक environment variable होता है जो JVM को बताता है कि .class files (bytecode) कहाँ locate करनी हैं।
- JVM जब किसी class को load करता है, तो वह CLASSPATH में दिए गए directories और JAR files में search करता है।
- यदि CLASSPATH सही set नहीं है, तो program compile तो हो सकता है पर run time पर "ClassNotFoundException" आती है।
- CLASSPATH में current directory (.) by default include होती है।

CLASSPATH set करने के तरीके :

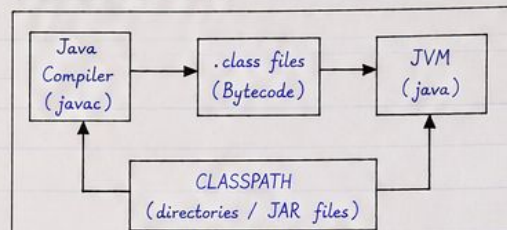
(i) Temporary (Command Prompt में)

```
set CLASSPATH=.;C:\MyClasses;C:\lib\mypack.jar
```

(ii) Permanent (Environment Variable में)

- System Properties → Environment Variables → New
- Variable name : CLASSPATH
- Variable value : .;C:\MyClasses;C:\lib\mypack.jar

CLASSPATH Working :



CLASSPATH में Multiple paths को Windows में ';' (semicolon) से और Linux / Mac में ':' (colon) से separate किया जाता है।

Example :

- Windows : .;C:\MyClasses;D:\lib\test.jar
- Linux / Mac : ./home/user/classes;/lib/mylib.jar

2. Auto Boxing and Unboxing :

- Java में primitive data types (int, double, char, etc.) और उनके wrapper classes (Integer, Double, Character, etc.) के बीच automatic conversion को Auto Boxing और Unboxing कहते हैं।
- Boxing : Primitive → Wrapper class
- Unboxing : Wrapper class → Primitive

Auto Boxing (Primitive → Wrapper)

```
int a = 10;
Integer obj = a; // Auto Boxing
// Compiler internally writes →
Integer obj = Integer.valueOf(a);
```

Unboxing (Wrapper → Primitive)

```
Integer obj = 20;
int b = obj; // Auto Unboxing
// Compiler internally writes →
int b = obj.intValue();
```

Note : Auto Boxing और Unboxing से code छोटा और readable बनता है।

Primitive and Wrapper Classes :

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
char	Character
float	Float
double	Double
boolean	Boolean

3. String in Java :

- String एक class है जो characters के sequence को store करती है।
- String objects immutable होते हैं, यानी एक बार create होने के बाद उसके value को बदला नहीं जा सकता।
- String constant pool में store होती है।

Declaration and Initialization :

```
String s1 = "Hello"; // String literal (Pool में)
String s2 = new String("Hello"); // new object (Heap में)
```

Important Methods :

```
String() → String की length प्राप्त करता है।
charAt(index) → index पर उपस्थित character देता है।
concat(str) → दो strings को जोड़ता है।
equals(str) → content compare करता है (case sensitive)
equalsIgnoreCase(str) → case ignore करके compare करता है।
toUpperCase() → सभी characters को upper case में करता है।
toLowerCase() → सभी characters को lower case में करता है।
```

Example :

```
String s1 = "Hello";
String s2 = "Java";
String s3 = s1.concat(" " + s2);
System.out.println(s3); // Hello Java
System.out.println(s1.length()); // 5
System.out.println(s1.charAt(1)); // e
System.out.println(s1.equals("hello")); // false
System.out.println(s1.equalsIgnoreCase("hello")); // true
```

4. StringBuffer in Java :

- StringBuffer एक mutable (परिवर्तनशील) class है।
- इसका उपयोग तब करते हैं जब बार-बार String को modify करना हो।
- यह thread-safe होती है (methods synchronized होते हैं)।
- यह String की तुलना में तेज होती है क्योंकि बार-बार new object create नहीं होता।

Declaration :

```
StringBuffer sb = new StringBuffer("Java");
```

Common Methods :

```
append(str) → String end में जोड़ता है।
insert(index, str) → index पर insert करता है।
delete(start, end) → start से end-1 तक delete करता है।
reverse() → String को reverse करता है।
capacity() → वर्तमान capacity बताता है।
```

Example :

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");
sb.insert(5, ",");
sb.delete(5, 6); // comma delete
sb.reverse();
System.out.println(sb); // dlroW olleH
```